from Table

Exercise 4

SQL Practice Exercise: Joins, Union, Filtering and Aggregate

Retrieve all employees and their assigned projects using a INNER JOIN

Return EmployeeID, FirstName, LastName, Department, Salary, ProjectID, ProjectName, Budget, Status.

```
Select EmployeeID, FirstName, LastName, Department, Salary,
       ProjectID, ProjectName, Budget, Status
from Employees As A
INNER JOIN Projects AS B
ON A.EmployeeID = B.EmployeeID;
```

EmployeeID	FirstName	LastName	Department	Salary	ProjectID	ProjectName	Budget	Status
1	John	Doe	IT	70000	101	AP Development	100000	Completed
1	John	Doe	Cybersecurity Audit	75000	103			
1	John	Doe	IT	70 000	103	Cybersecurity Audit	75000	Pending
2	Alice	Smith	HR	60000	102	Employee Training	50000	Ongoing
3	Bob	Johnson	Finance	75000	104	Financial Analysis	90000	Ongoing
5	Emma	Wilson	Sales	65000	105	Market Expansion	65000	Completed
6	Michael	Clark	Finance	80000	106	Risk Management	80000	Pending

employee projects, including

2 Retrieve all employees and their assigned projects, including employees who have no projects using a left JOIN:

```
Select EmployeeID, FirstName, LastName, Department, Salary, ProjectID,
       ProjectName, Budget, Status
FROM Employees AS A
LEFT JOIN Projects As B
ON A employeeID = B employeeID;
```

EmployeeID	FirstName	LastName	Department	Salary	ProjectID	ProjectName	Budget	Status
1	John	Doe	IT	70000	101	AI Development	100000	Completed
1	John	Doe	IT	70000	103	Cybersecurity	75000	Pending
2	Alice	Smith	HR	6000	102	Employee Training	50000	Ongoing
3	Bob	Johnson	Finance	75000	104	Financial Analysis	90000	Ongoing
4	David	Brown	IT	72000	NULL	NULL	NULL	NULL
5	Emma	Wilson	Sales	65000	105	Market Expansion	65000	Completed
6	Michael	Clark	Finance	80000	106	Risk Management	80000	Pending

3 Retrieve all projects and their assigned employees, including projects that have no employees using a Right Join

```
Select ProjectID, ProjectName, Budget, Status, EmployeeID, FirstName,
       LastName, Department, Salary
FROM Projects As A
RIGHT JOIN Employees As B
ON AemployeeID = BEmployeeID;
```

ProjectID	ProjectName	Budget	EmployeeID	FirstName	LastName	Department	Salary	Status
101	AI Development	100000	1	John	Doe	IT	70000	Completed
102	Employee Training	50000	2	Alice	Smith	HR	60000	Ongoing
103	Cybersecurity Audit	75000	1	John	Doe	IT	70000	Pending
104	Financial Analysis	90000	3	Bob	Johnson	Finance	75000	Ongoing
105	Market Expansion	65000	5	Emma	Wilson	Sales	65000	Completed
106	Risk Management	80000	6	Michael	Clark	Finance	80000	Pending

4 Retrieve all employees and projects, including those without a match in either table using a FULL OUTER JOIN

```
Select EmployeeID, FirstName, LastName, Department, Salary, ProjectID,
ProjectName, Budget, Status
FROM Employees AS A
FULL OUTER JOIN Projects AS B
ON A employeeID = B employeeID;
```

EmployeeID	FirstName	LastName	Department	Salary	ProjectID	ProjectName	Budget	Status
1	John	Doe	IT	70000	101	AI Development	100000	Completed
2	Alice	Smith	HR	60000	102	Employee Training	50000	Ongoing
3	Bob	Johnson	Finance	75000	104	Financial Analysis	90000	Ongoing
4	David	Brown	IT	72000	NULL	NULL	NULL	NULL
5	Emma	Wilson	Sales	65000	105	Market Expansion	65000	Completed
6	Michael	Clark	Finance	80000	106	Risk Management	80000	Pending
1	John	Doe	IT	70000	103	Cybersecurity Audit	75000	Pending

filtering Statements

7 Retrieve employees who earn more than 70 000

```
Select EmployeeID, FirstName, LastName, Department, Salary
from employees
Where Salary > 70 000 ;
```

EmployeeID	FirstName	LastName	Department	Salary
3	Bob	Johnson	Finance	75 000
4	David	Brown	IT	72 000
6	Michael	clark	finance	80 000

8 Retrieve employees working in either IT or Finance departments.

```
Select EmployeeID, FirstName, LastName, Department, Salary
from employees
Where department = 'IT' OR 'Finance' Department = 'Finance';
```

EmployeeID	FirstName	LastName	Department	Salary
1	John	Doe	IT	70 000
3	Bob	Johnson	Finance	75 000
4	David	Brown	IT	72 000
6	Michael	clark	Finance	80 000

9 Retrieve projects that are not yet completed

```
Select ProjectID, ProjectName, Budget, Status
from projects
Where Status != completed ;
```

ProjectID	ProjectName	Budget	Status
102	Employee Training	50 000	Ongoing
103	Cybersecurity Audit	75000	Pending
104	Financial Analysis	90 000	Ongoing
106	Risk Management	80 000	Pending

10 Retrieve projects that have a budget greater than 70 000 and are not completed.

```
Select ProjectID, ProjectName, Budget, Status
from projects
Where budget > 70000 AND Status != Completed;
```

ProjectID	ProjectName	Budget	Status
103	Cybersecurity Audit	75000	Pending
104	Financial Analysis	90000	Ongoing
106	Risk Management	80 000	Pending

11 Retrieve employees from New York or Toronto, ordered by Salary in descending order

```
Select EmployeeID, FirstName, LastName, Department, Salary, City
From employees
Where City = 'New York' or City = Toronto
GROUP BY Salary
Order By Salary Desc;
```

EmployeeID	FirstName	LastName	Department	Salary	City
6	Michael	Clark	Finance	80 000	New York
3	Bob	Johnson	Finance	75000	Toronto
1	John	Doe	IT	70 000	New York

12 Retrieve the top 3 highest-paid employees

```
Select EmployeeID, FirstName, LastName, Department, Salary
from employees
ORDER BY Salary Desc,
Limit 3;
```

EmployeeID	FirstName	LastName	Department	Salary
6	Michael	Clark	Finance	80 000
3	Bob	Johnson	Finance	75 000
4	David	Brown	IT	72000

Aggregate functions with GROUP BY & HAVING

13 Find the total salary per department, sorted in descending order

```
Select Department, Sum(Salary) As 'TotalSalary'
From employees
GROUP BY Department
ORDER BY 'TotalSalary' Desc;
```

Department	TotalSalary
finance	155000
IT	142000
Sales	65000
HR	60000

14- Find the average salary per city, but only include cities where the average salary is greater than 65 000

```
Select City, Avg(Salary) As 'AverageSalary'
From employees
GROUP BY City
Having 'AverageSalary' > 65000 ;
```

City	AverageSalary
New York	75 000
Toronto	75 000
London	72 000

15 Count the number of employees per department, including only departments with more than 1 employee

~~Select department, Count(*) As emp~~

```
Select department, Count(EmployeeID) As 'Employee-Count'
from employees
GROUP BY department
Having 'EmployeeCount' > 1 ;
```

Department	EmployeeCount
IT	2
Finance	2

16 Retrieve the number of projects per status, but only include statuses with at least 2 projects

```
Select status, Count(ProjectID) As EmployeeCount
From projects
GROUP BY status
Having 'employeeCount' >= 2;
```

Status	EmployeeCount
Completed	2
Ongoing	2
Pending	2

17 Retrieve the total project budget per employee, but only for employees who are managing projects worth more than 150 000

```
Select EmployeeID, FirstName, LastName, Sum(Budget) As totalProjectBudget
From projects, employees
GROUP BY Proj Budget
Having ProjectBudget > 150 000;
```

EmployeeID	FirstName	LastName	ProjectBudget
1	John	Doe	175 000